C Dependency Generation part 1 - GNU Make

Advanced Embedded Software Development

with **Dan Walkes**


University of Colorado **Boulder**

Learning objectives:

How to handle incremental builds with GNU make.

How to automate makefile generation with dependency tracking using GNU make.

GNU Make Dependencies

- When a source file references header files:
  - The object file needs to be rebuilt when the header(s) **or** source file changes
  - You can do this by adding header files as prerequisites to your make targets
    - Works for small projects
  - You can also use gcc and gnu make to generate these automatically

https://stackoverflow.com/questions/8025766/makefile-auto-dependency-generation

Simple Dependencies Project

main.c

```c
#include "file1.h"
#include "file2.h"
#include <stdio.h>

int main(int argc, char**argv)
{
        printf("Hello World!\n");
        file1_hello();
        file2_hello();
        return 0;
}
```

file1.c, file2.c

```c
#include "file1.h"
#include <stdio.h>

void file1_hello()
{
        printf("Hello from file1.c!\n");
}
```

```c
#include "file2.h"
#include <stdio.h>

void file2_hello()
{
        printf("Hello from file2.c!\n");
}
```

file1.h, file2.h

```c
1   void file1_hello();
```

```c
1   void file2_hello();
```

https://github.com/cu-ecen-5013/auto-dependency-generation-example

GCC Dependency Build Support

- We can use GCC and GNU Make to build dependencies for us
  - GCC can generate GNU Make compatible dependency files with command options
    - -MMD - Generate dependency files
    - -MT - Set the target name
    - -MF - Where to write
  - Not an original GCC feature - was previously done with sed commands

https://github.com/cu-ecen-5013/auto-dependency-generation-example
http://make.mad-scientist.net/papers/advanced-auto-dependency-generation/
https://gcc.gnu.org/onlinedocs/gcc-5.2.0/gcc/Preprocessor-Options.html

GCC Dependency Build Support

```
ecen5013@ecen5013-VirtualBox:~/auto-dependency-generation-example$ cat .deps/file1.d
file1.o: file1.c file1.h

file1.h:
ecen5013@ecen5013-VirtualBox:~/auto-dependency-generation-example$ cat .deps/file2.d
file2.o: file2.c file2.h

file2.h:
ecen5013@ecen5013-VirtualBox:~/auto-dependency-generation-example$ cat .deps/main.d
main.o: main.c file1.h file2.h

file1.h:

file2.h:
```

- Example generated dependency files

https://github.com/cu-ecen-5013/auto-dependency-generation-example

GNU Makefile Dependency Setup

```
# -- TODO: customize the list below for your project ---
# List of source .c files used with the project
SRCS := main.c file1.c file2.c

# The aplication generated
APPNAME = depend-generation-test
# -- End of customization section ---

# Replace .c extension on SRCS to get objfiles using gnu make pattern rules and substitution references.
# See https://www.gnu.org/software/make/manual/html_node/Pattern-Intro.html#Pattern-Intro for pattern rules and
# https://www.gnu.org/software/make/manual/html_node/Substitution-Refs.html#Substitution-Refs for substitution references overview
OBJFILES := $(SRCS:%.c=%.o)

# Build the app you've specified in APPNAME for the "all" or "default" target
all : $(APPNAME)
default : $(APPNAME)

# Build the application by running the link step with all objfile inputs
$(APPNAME) : $(OBJFILES)
        $(CC) $(LDFLAGS) $^ -o $(APPNAME)
```

https://github.com/cu-ecen-5013/auto-dependency-generation-example
http://make.mad-scientist.net/papers/advanced-auto-dependency-generation/

GNU Makefile Dependency Setup

```
# The directory (hidden) where dependency files will be stored
DEPDIR := .deps
# Flags passed to gcc to automatically build dependencies when compiling
# See https://www.gnu.org/software/make/manual/html_node/Automatic-Variables.html for detail about variable names
# $@ references the target file of the rule and will be "main.o" when compiling "main.c"
# $* references the stem of the rule, and will be "main" when target is "main.o"
DEPFLAGS = -MT $@ -MMD -MP -MF $(DEPDIR)/$*.d
```

```
# Rules for compiling a C file, including DEPFLAGS along with Implicit GCC variables.
# See https://www.gnu.org/software/make/manual/html_node/Implicit-Variables.html
# and see https://www.gnu.org/software/make/manual/html_node/Catalogue-of-Rules.html#Catalogue-of-Rules
# for the default c rule
COMPILE.c = $(CC) $(DEPFLAGS) $(CFLAGS) $(CPPFLAGS) -c
```

https://github.com/cu-ecen-5013/auto-dependency-generation-example
http://make.mad-scientist.net/papers/advanced-auto-dependency-generation/

GNU Makefile Dependency Setup

```
# Delete the built-in rules for building object files from .c files
%.o : %.c
# Define a rule to build object files based on .c or dependency files by making the associated dependency file
# a prerequisite of the target.  Make the DEPDIR an order only prerequisite of the target, so it will be created when needed, meaning
# the targets won't get rebuilt when the timestamp on DEPDIR changes
# See https://www.gnu.org/software/make/manual/html_node/Prerequisite-Types.html for order only prerequesites overview.
%.o : %.c $(DEPDIR)/%.d | $(DEPDIR)
	$(COMPILE.c) $(OUTPUT_OPTION) $<
```

```
# Use pattern rules to build a list of DEPFILES
DEPFILES := $(SRCS:%.c=$(DEPDIR)/%.d)
# Mention each of the dependency files as a target, so make won't fail if the file doesn't exist
$(DEPFILES):

# Include all dependency files which exist, to include the relevant targets.
# See https://www.gnu.org/software/make/manual/html_node/Wildcard-Function.html for wildcard function documentation
include $(wildcard $(DEPFILES))
```

https://github.com/cu-ecen-5013/auto-dependency-generation-example
http://make.mad-scientist.net/papers/advanced-auto-dependency-generation/

GNU Makefile Dependency Testing

```
ecen5013@ecen5013-VirtualBox:~/auto-dependency-generation-example$ ./test.sh
make output after touching file1.h was:
cc -MT main.o -MMD -MP -MF .deps/main.d -Wall -Werror  -c -o main.o main.c
cc -MT file1.o -MMD -MP -MF .deps/file1.d -Wall -Werror  -c -o file1.o file1.c
cc  main.o file1.o file2.o -o depend-generation-test
make output after touching file2.h was:
cc -MT main.o -MMD -MP -MF .deps/main.d -Wall -Werror  -c -o main.o main.c
cc -MT file2.o -MMD -MP -MF .deps/file2.d -Wall -Werror  -c -o file2.o file2.c
cc  main.o file1.o file2.o -o depend-generation-test
make output after touching main.c was:
cc -MT main.o -MMD -MP -MF .deps/main.d -Wall -Werror  -c -o main.o main.c
cc  main.o file1.o file2.o -o depend-generation-test
```

- See test.sh script test case

https://github.com/cu-ecen-5013/auto-dependency-generation-example
https://github.com/cu-ecen-5013/auto-dependency-generation-example/blob/master/test.sh

Pitfalls of GNU Make

- Dependency tracking isn't automatic
  - Some dependencies are still missing (for instance Makefile) in the project I shared.
- Makefile implementation details are typically platform specific
  - Assumes a specific compiler
  - Assumes binary utilities are present in the root filesystem
  - Means your code won't build on other platforms

https://www.gnu.org/savannah-checkouts/gnu/autoconf/manual/autoconf-2.69/autoconf.html#Automake
https://www.gnu.org/savannah-checkouts/gnu/autoconf/manual/autoconf-2.69/autoconf.html#Introduction

C Dependency Generation part 2 - Automake

Advanced Embedded Software Development

with **Dan Walkes**


University of Colorado **Boulder**

Learning objectives:

Introduce Autoconf and Automake

Understand how to use Autoconf and Automake to perform dependency generation.

Autoconf and Automake

- Autoconf was created in 1991
- Produces configure scripts for building, installing and packaging software
- Implementation leverages BASH scripts running on the host build system

https://en.wikipedia.org/wiki/Autoconf

Autoconf and Automake

- Uses a configure.ac file which describes your project
- Generates a “configure” script which:
  - Automatically generates a makefile with correct dependencies
  - May generate a header file containing #define directives with common types/formats
    - Configures build for a specific platform
- ./configure && make && sudo make install

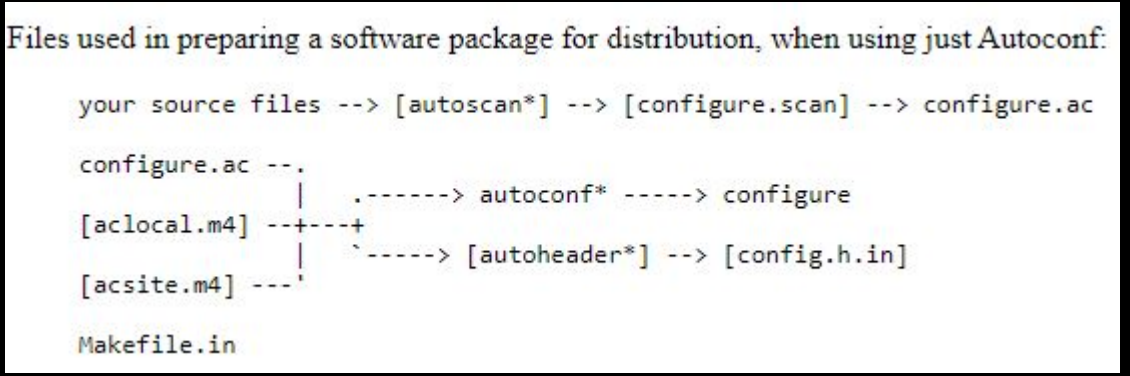
https://www.gnu.org/savannah-checkouts/gnu/autoconf/manual/autoconf-2.69/autoconf.html#Making-configure-Scripts

Autoconf and Automake

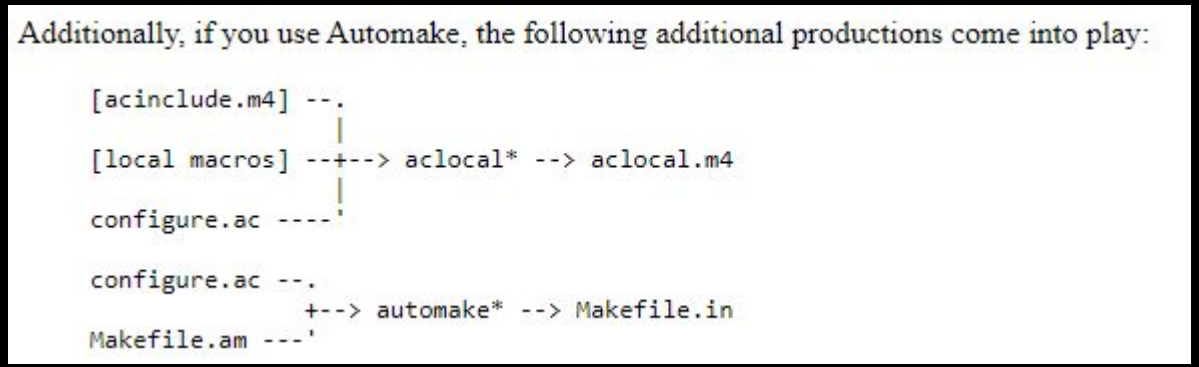
Programs that are executed are suffixed by '*'.

Optional files are enclosed in square brackets ('[]').

Files used in preparing a software package for distribution, when using just Autoconf:



Additionally, if you use Automake, the following additional productions come into play:



https://www.gnu.org/savannah-checkouts/gnu/autoconf/manual/autoconf-2.69/autoconf.html#Making-configure-Scripts

Autoconf and Automake - Simpler Implementation

- Autoconf and Automake interact in non-obvious ways, have evolved independently.
- Documentation covers lots of possible use-case, is a bit intimidating.
- Other tutorials outline a simpler scheme for simple projects, we will follow this one in particular:
  http://www.idryman.org/blog/2016/03/10/autoconf-tutorial-1/

http://www.idryman.org/blog/2016/03/10/autoconf-tutorial-1/

do-autoconf-automake-setup.sh

```
# TODO: Configure the section below for your application
appname="automaketest"
version="1.0"
sources="main.c file1.c file2.c"
# --- End of configuration section
```

```
cat << EOF > configure.ac
# See https://www.gnu.org/savannah-checkouts/gnu/autoconf/manual/autoconf-2.69/autoconf.html#Initializing-configure
AC_INIT([${appname}],[${version}])
# See https://www.gnu.org/software/automake/manual/html_node/Public-Macros.html
# and see https://www.gnu.org/software/automake/manual/html_node/List-of-Automake-options.html#List-of-Automake-options for options
# -Wall and -Werror tells automake to use these compiler flags when building the makefile (see https://www.gnu.org/software/automake/manual
# foreign says we aren't going to conform to gnu coding standard (with README, ChangeLog, AUTHORS and other required files in the home dire
AM_INIT_AUTOMAKE([-Wall -Werror foreign])
# Find a valid C compiler
AC_PROG_CC
# Tell automake to create the Makefile for us
# See https://www.gnu.org/software/automake/manual/html_node/Requirements.html
AC_CONFIG_FILES([Makefile])
# Tell autoconf to create the ./configure script
AC_OUTPUT
EOF
```

https://github.com/cu-ecen-5013/autoconf-automake-example/blob/master/do-autoconf-automake-setup.sh

do-autoconf-automake-setup.sh

```
# TODO: Configure the section below for your application
appname="automaketest"
version="1.0"
sources="main.c file1.c file2.c"
# --- End of configuration section
```

```
cat << EOF > Makefile.am
# A list of all programs generated with this project
# The bin_ prefix means the install of these programs will be to the bindir directory (typically /bin) when running the make install target
bin_PROGRAMS = ${appname}
# A list of source files to use when generating the makefile
${appname}_SOURCES = ${sources}

# Add make syntax test target to run our travis tests
# See https://stackoverflow.com/questions/1500039/adding-additional-make-test-targets-to-an-autotools-project for details
test: ; @./test.sh
.PHONY: test
EOF
```

```
# See https://www.gnu.org/software/autoconf/manual/autoconf-2.68/html_node/autoreconf-Invocation.html
# Update configure scripts with verbose, install (add-missing) and force arguments to setup a configure
# script with latest content of configure.ac and Makefile.am created above
autoreconf -vif
```

https://github.com/cu-ecen-5013/autoconf-automake-example/blob/master/do-autoconf-automake-setup.sh

configure example

```
ecen5013@ecen5013-VirtualBox:~/autoconf-automake-example$ ./configure
checking for a BSD-compatible install... /usr/bin/install -c
checking whether build environment is sane... yes
checking for a thread-safe mkdir -p... /bin/mkdir -p
checking for gawk... gawk
checking whether make sets $(MAKE)... yes
checking whether make supports nested variables... yes
checking for gcc... gcc
checking whether the C compiler works... yes
checking for C compiler default output file name... a.out
checking for suffix of executables...
checking whether we are cross compiling... no
checking for suffix of object files... o
checking whether we are using the GNU C compiler... yes
checking whether gcc accepts -g... yes
checking for gcc option to accept ISO C89... none needed
checking whether gcc understands -c and -o together... yes
checking for style of include used by make... GNU
checking dependency style of gcc... gcc3
checking that generated files are newer than configure... done
configure: creating ./config.status
config.status: creating Makefile
config.status: executing depfiles commands
```

- After do-autoconf-automake-setup.sh we now have a configure script
- Run this to create our Makefile

https://github.com/cu-ecen-5013/autoconf-automake-example/blob/master/do-autoconf-automake-setup.sh

Generated Makefile

configure.ac | Makefile

```
358     -rm -f *.tab.c
359
360 include ./$(DEPDIR)/file1.Po
361 include ./$(DEPDIR)/file2.Po
362 include ./$(DEPDIR)/main.Po
363
364 .c.o:
365     $(AM_V_CC)$(COMPILE) -MT $@ -MD -MP -MF $(DEPDIR)/$*.Tpo -c -o $@ $<
366     $(AM_V_at)$(am__mv) $(DEPDIR)/$*.Tpo $(DEPDIR)/$*.Po
367 #   $(AM_V_CC)source='$<' object='$@' libtool=no \
368 #   DEPDIR=$(DEPDIR) $(CCDEPMODE) $(depcomp) \
369 #   $(AM_V_CC_no)$(COMPILE) -c -o $@ $<
370
```

```
266 top_builddir = .
267 top_srcdir = .
268 # A list of source files to use when generating the makefile
269 automaketest_SOURCES = main.c file1.c file2.c
270 all: all-am
271
```

- ./configure step generated a makefile for us with dependencies

Autoconf/Automake and Buildroot

- Natively supported with buildroot and $(eval $(autotools-package))
  - Will automatically configure for cross compilation
  - Will use install target commands to install to buildroot/output/target directory

https://buildroot.org/downloads/manual/manual.html#_infrastructure_for_autotools_based_packages

Autoconf/Automake Pitfalls

- Can be intimidating to set up
- Extensive documentation, very complex tool, varied use-cases
- Carries some historical baggage
- Tailored for gnu software defined workflows
- Not well supported on non Unix/Linux variants (IE Windows)

https://en.wikipedia.org/wiki/Autoconf

C Dependency Generation part 3 - CMake

Advanced Embedded Software Development

with **Dan Walkes**


University of Colorado **Boulder**

Learning objectives:

Introduce CMake

Understand how to use CMake to perform dependency generation.

Comparison of GNU Make/Automake/CMake tools.

CMake

- Cross platform open source system to manage build processes in a compiler independent manner
- CMakeLists.txt files containing configuration information about the project
- Generates standard build files for the platform:
  - GNU makefiles on Linux/Unix
  - Microsoft Visual Studio Projects/Workspaces on Windows
    - Also works natively with recent Visual Studio versions

https://cmake.org/

CMake

```
1  cmake_minimum_required(VERSION 3.0.0)
2  project(cmake-example)
3  add_executable(cmake-example file1.c file2.c main.c)
```

- CMakeLists.txt
  - List minimum required version for cmake
  - Add project name
  - Add executable name and required files
- Install cmake
  - sudo apt install cmake
  - Or download cmake installer at

    https://cmake.org/download/

https://github.com/cu-ecen-5013/cmake-example
https://github.com/cu-ecen-5013/cmake-example/blob/master/CMakeLists.txt

CMake

- Create a build directory
  - This is the location where build files will be generated for your platform.
- Execute cmake with relative path to location of CMakeLists.txt
  - This generates build files (ie makefiles)
- Run make
- Run your application

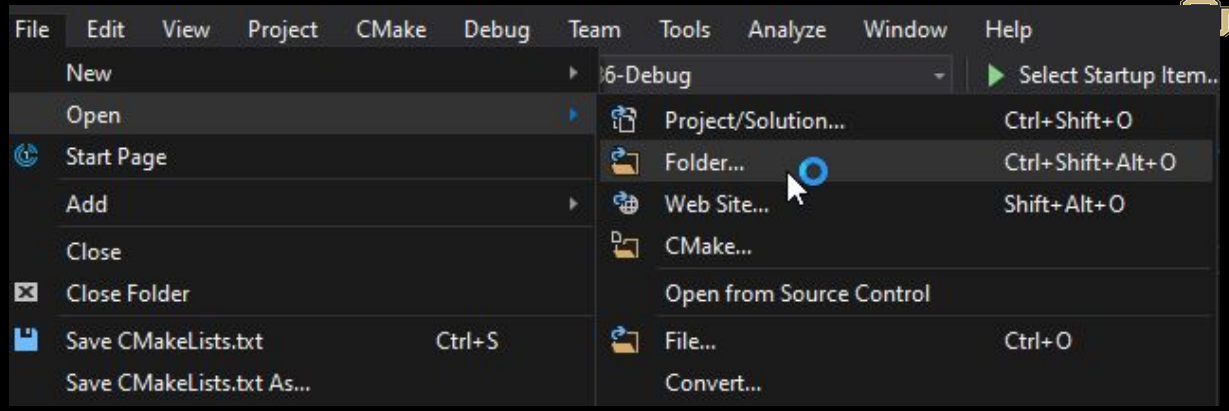
https://github.com/cu-ecen-5013/cmake-example

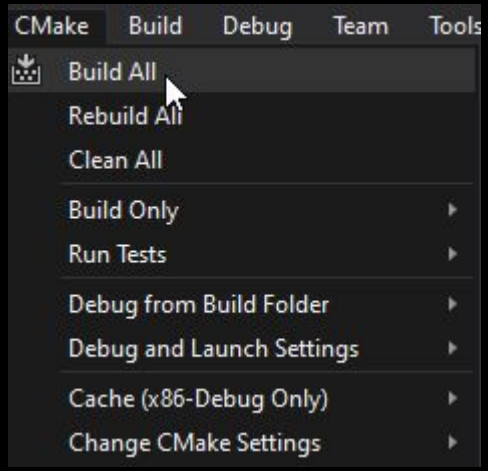
CMake on Linux

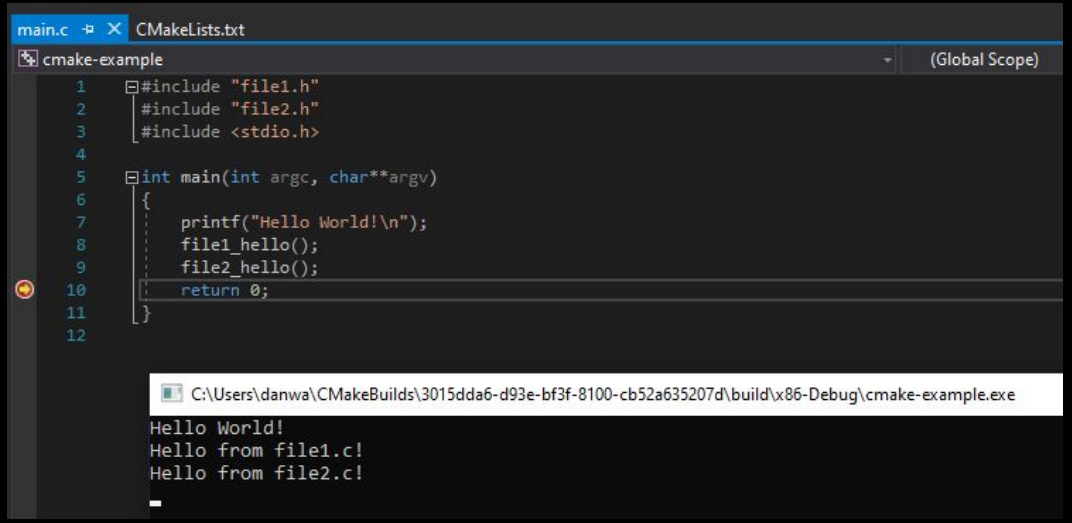
```
ecen5013@ecen5013-VirtualBox:~/cmake-example$ mkdir build
ecen5013@ecen5013-VirtualBox:~/cmake-example$ cd build
ecen5013@ecen5013-VirtualBox:~/cmake-example/build$ cmake ..
-- The C compiler identification is GNU 7.4.0
-- The CXX compiler identification is GNU 7.4.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/ecen5013/cmake-example/build
ecen5013@ecen5013-VirtualBox:~/cmake-example/build$ ls
CMakeCache.txt  CMakeFiles  cmake_install.cmake  Makefile
ecen5013@ecen5013-VirtualBox:~/cmake-example/build$ make
Scanning dependencies of target cmake-example
[ 25%] Building C object CMakeFiles/cmake-example.dir/file1.c.o
[ 50%] Building C object CMakeFiles/cmake-example.dir/file2.c.o
[ 75%] Building C object CMakeFiles/cmake-example.dir/main.c.o
[100%] Linking C executable cmake-example
[100%] Built target cmake-example
ecen5013@ecen5013-VirtualBox:~/cmake-example/build$ ./cmake-example
Hello World!
Hello from file1.c!
Hello from file2.c!
```

https://github.com/cu-ecen-5013/cmake-example

CMake Windows







https://github.com/cu-ecen-5013/cmake-example
https://docs.microsoft.com/en-us/cpp/build/cmake-projects-in-visual-studio?view=vs-2017
https://docs.microsoft.com/en-us/cpp/linux/download-install-and-setup-the-linux-development-workload?view=vs-2019

CMake and Buildroot

- Natively supported with
  - $(eval $(cmake-package)) in package.mk file

https://buildroot.org/downloads/manual/manual.html#_infrastructure_for_cmake_based_packages

Build Tool Comparisons

	GNU Make	Autoconf/Autotools	CMake
Cross Platform Support	Limited	Better	Best
Configuration Complexity	Complex	Easier (arguably)	Easiest (arguably)